

Name: Olorunfemi Emmanuel Adeleke

Matric No.:241201061

Level:200

Department: Computer Science

Course Code: COS 202

Assignment solutions

1.) Calculator

```
import tkinter as tk
from tkinter import messagebox

class MathematicalCalculator:
    def __init__(self, root):
        self.root = root
        self.root.title("Mathematical Calculator")
        self.root.geometry("380x550")
        self.root.configure(bg="#202020")
        self.root.resizable(False, False)

        self.expression = ""

        # Screen Display String Variable
        self.equation = tk.StringVar()

        self.create_widgets()

    def create_widgets(self):
        # Display Screen Container
        display_frame = tk.Frame(self.root, width=380, height=100, bd=0,
bg="#202020")
        display_frame.pack(side=tk.TOP, fill=tk.BOTH)

        # Output Display Screen
        display_field = tk.Entry(
            display_frame,
            textvariable=self.equation,
```

```

        font=('Arial', 28, 'bold'),
        fg="#FFFFFF",
        bg="#202020",
        bd=0,
        justify=tk.RIGHT,
        insertbackground='white'
    )
    display_field.pack(side=tk.TOP, fill=tk.BOTH, ipadx=8, ipady=30, padx=10,
pady=10)

```

```

# Keypad Container
button_frame = tk.Frame(self.root, width=380, height=450,
bg="#202020")
button_frame.pack()

```

```

# Grid configuration for uniform sizing
for i in range(5):
    button_frame.rowconfigure(i, weight=1)
for j in range(4):
    button_frame.columnconfigure(j, weight=1)

```

```

# Button layout definition: (Text, Row, Column, Text Color, Background
Color)

```

```

buttons = [
    ('C', 0, 0, '#FF3B30', '#3A3A3A'), ('^', 0, 1, '#00D2FF', '#3A3A3A'),
    ('%', 0, 2, '#00D2FF', '#3A3A3A'), ('/', 0, 3, '#00D2FF', '#FF9500'),

    ('7', 1, 0, '#FFFFFF', '#2E2E2E'), ('8', 1, 1, '#FFFFFF', '#2E2E2E'),
    ('9', 1, 2, '#FFFFFF', '#2E2E2E'), ('*', 1, 3, '#00D2FF', '#FF9500'),

    ('4', 2, 0, '#FFFFFF', '#2E2E2E'), ('5', 2, 1, '#FFFFFF', '#2E2E2E'),
    ('6', 2, 2, '#FFFFFF', '#2E2E2E'), ('-', 2, 3, '#00D2FF', '#FF9500'),

    ('1', 3, 0, '#FFFFFF', '#2E2E2E'), ('2', 3, 1, '#FFFFFF', '#2E2E2E'),
    ('3', 3, 2, '#FFFFFF', '#2E2E2E'), ('+', 3, 3, '#00D2FF', '#FF9500'),

    ('OFF', 4, 0, '#FF3B30', '#3A3A3A'), ('0', 4, 1, '#FFFFFF', '#2E2E2E'),
    ('\ \', 4, 2, '#00D2FF', '#3A3A3A'), ('=', 4, 3, '#FFFFFF', '#FF9500')
]

```

```

# Generate Buttons dynamically

```

```

for text, row, col, fg_color, bg_color in buttons:
    action = lambda x=text: self.on_button_click(x)

    btn = tk.Button(
        button_frame,
        text=text,
        fg=fg_color,
        bg=bg_color,
        font=('Arial', 18, 'bold'),
        bd=0,
        activebackground='#4A4A4A',
        activeforeground='#FFFFFF',
        cursor="hand2",
        command=action
    )
    btn.grid(row=row, column=col, sticky="nsew", padx=4, pady=4,
ipadx=10, ipady=15)

def on_button_click(self, char):
    if char == 'C':
        self.expression = ""
        self.equation.set("")

    elif char == 'OFF':
        if messagebox.askokcancel("Quit", "Do you want to turn off the
calculator?"):
            self.root.destroy()

    elif char == '=':
        try:
            # Replace visual operators with valid Python operators
            # '\' represents Integer Division (//)
            # '^' represents Exponentiation (**)
            eval_expression = self.expression.replace('\ ', '//').replace('^', '**')

            # Prevent evaluation of empty strings
            if eval_expression.strip() == "":
                return

            result = str(eval(eval_expression))
            self.equation.set(result)

```

```

        self.expression = result # Allow continuing calculations on result

    except ZeroDivisionError:
        messagebox.showerror("Error", "Cannot divide by zero")
        self.expression = ""
        self.equation.set("")
    except Exception:
        messagebox.showerror("Error", "Invalid Expression")
        self.expression = ""
        self.equation.set("")

    else:
        self.expression += str(char)
        self.equation.set(self.expression)

# Application initialization
if __name__ == "__main__":
    root = tk.Tk()
    app = MathematicalCalculator(root)
    root.mainloop()

```

2.) Personal Pocket Cgpa calculator (PPC)

```

import tkinter as tk
from tkinter import messagebox

class PocketCGPACalculator:
    def __init__(self, root):
        self.root = root
        self.root.title("PPC")
        self.root.geometry("360x540")
        self.root.configure(bg="#0F172A") # Premium Dark Slate Theme
        self.root.resizable(False, False)

    # Internal Database
    self.total_units = 0

```

```

self.total_points = 0

# Header Graphic
tk.Label(self.root, text=" POCKET CGPA (PPC)", font=("Helvetica", 14, "bold"),
bg="#2563EB", fg="#FFFFFF", pady=12).pack(fill="x")

# Input Workspace
self.frame = tk.Frame(self.root, bg="#0F172A", padx=20, pady=15)
self.frame.pack(fill="both", expand=True)

# Field: Course Title
tk.Label(self.frame, text="Course Code:", bg="#0F172A", fg="#94A3B8",
font=("Helvetica", 10, "bold")).pack(anchor="w", pady=(5,0))
self.code_input = tk.Entry(self.frame, bg="#1E293B", fg="#FFFFFF", bd=0,
insertbackground="white", font=("Helvetica", 12))
self.code_input.pack(fill="x", ipady=8, pady=5)

# Field: Total Score
tk.Label(self.frame, text="Score (0 - 100):", bg="#0F172A", fg="#94A3B8",
font=("Helvetica", 10, "bold")).pack(anchor="w", pady=(5,0))
self.score_input = tk.Entry(self.frame, bg="#1E293B", fg="#FFFFFF", bd=0,
insertbackground="white", font=("Helvetica", 12))
self.score_input.pack(fill="x", ipady=8, pady=5)

# Field: Credit Units
tk.Label(self.frame, text="Credit Units:", bg="#0F172A", fg="#94A3B8",
font=("Helvetica", 10, "bold")).pack(anchor="w", pady=(5,0))
self.unit_input = tk.Entry(self.frame, bg="#1E293B", fg="#FFFFFF", bd=0,
insertbackground="white", font=("Helvetica", 12))
self.unit_input.pack(fill="x", ipady=8, pady=5)

# Add Button
tk.Button(self.frame, text="Log Course", bg="#10B981", fg="#FFFFFF",
activebackground="#059669", font=("Helvetica", 11, "bold"), bd=0,
cursor="hand2", command=self.process_entry).pack(fill="x", ipady=10,
pady=20)

# Live Display Dash
self.dash = tk.Frame(self.frame, bg="#1E293B", bd=0, padx=15, pady=15)
self.dash.pack(fill="x", pady=10)

```

```

        self.cgpa_label = tk.Label(self.dash, text="CGPA: 0.00", font=("Helvetica",
20, "bold"), bg="#1E293B", fg="#F59E0B")
        self.cgpa_label.pack()

        self.class_label = tk.Label(self.dash, text="Standing: Expecting Data",
font=("Helvetica", 10, "italic"), bg="#1E293B", fg="#94A3B8")
        self.class_label.pack(pady=(5,0))

        # Clear System Link
        tk.Button(self.frame, text="Reset Pocket Log", bg="#EF4444", fg="#FFFFFF",
font=("Helvetica", 9), bd=0, cursor="hand2",
command=self.reset_system).pack(pady=5)

    def process_entry(self):
        """Validates entry states and parses data structures using conditional
checks."""
        raw_code = self.code_input.get().strip()
        raw_score = self.score_input.get().strip()
        raw_unit = self.unit_input.get().strip()

        # Guard Clause: Selection rule for empty entries
        if not raw_code or not raw_score or not raw_unit:
            messagebox.showwarning("Missing Input", "Fill out all pocket entry cells.")
            return

        try:
            score = float(raw_score)
            units = int(raw_unit)
        except ValueError:
            messagebox.showerror("Format Error", "Score and Units must be pure
numerical metrics.")
            return

        # Selection Control: Out-of-bounds validations
        if score < 0 or score > 100:
            messagebox.showerror("Data Error", "Academic assessment scores must
fall between 0 and 100.")
            return
        if units <= 0:
            messagebox.showerror("Data Error", "Credit unit weights must be greater
than zero.")

```

```
return
```

```
# Selection Control Statements: Grade Point Evaluator
```

```
if score >= 70:
```

```
    grade, gp = "A", 5
```

```
elif score >= 60:
```

```
    grade, gp = "B", 4
```

```
elif score >= 50:
```

```
    grade, gp = "C", 3
```

```
elif score >= 45:
```

```
    grade, gp = "D", 2
```

```
elif score >= 40:
```

```
    grade, gp = "E", 1
```

```
else:
```

```
    grade, gp = "F", 0
```

```
# Accumulate metrics
```

```
self.total_units += units
```

```
self.total_points += (gp * units)
```

```
# Calculate current cumulative scale
```

```
cgpa = self.total_points / self.total_units
```

```
# Selection Control Statements: Class of Degree Mapping
```

```
if cgpa >= 4.50:
```

```
    standing = "First Class Honours "
```

```
elif cgpa >= 3.50:
```

```
    standing = "Second Class (Upper Division)"
```

```
elif cgpa >= 2.40:
```

```
    standing = "Second Class (Lower Division)"
```

```
elif cgpa >= 1.50:
```

```
    standing = "Third Class Honours"
```

```
elif cgpa >= 1.00:
```

```
    standing = "Pass"
```

```
else:
```

```
    standing = "Fail (Academic Probation)"
```

```
# Update Dashboard Display Graphics
```

```
self.cgpa_label.config(text=f"CGPA: {cgpa:.2f}")
```

```
self.class_label.config(text=f"Standing: {standing}")
```

```

        # Feedback confirmation toast
        messagebox.showinfo("Success", f"{raw_code.upper()} Logged!\nGrade:
{grade} (GP: {gp})")

    # Wipe fields clean for next submission process
    self.code_input.delete(0, tk.END)
    self.score_input.delete(0, tk.END)
    self.unit_input.delete(0, tk.END)
    self.code_input.focus()

def reset_system(self):
    """Wipes tracking registers back to zero configurations."""
    self.total_units = 0
    self.total_points = 0
    self.cgpa_label.config(text="CGPA: 0.00")
    self.class_label.config(text="Standing: Expecting Data")
    messagebox.showinfo("Reset Complete", "All tracking records cleared.")

if __name__ == "__main__":
    root = tk.Tk()
    app = PocketCGPACalculator(root)
    root.mainloop()

```